stackPRS — a disease-specific ensemble on top of pretrained PRS

Logic. Pretrained polygenic risk scores are pulled from the PGS Catalog and computed for every individual with pgsc_calc. Those scores are the features. An ensemble is then fitted on top of them, a kind of fine-tuning for one disease:

```
PGS Catalog (~7,000 pretrained PRS)
   │  step 02: leakage · ancestry · phenotype filters
   ▼
pgsc_calc on your genotypes → [N individuals × selected PRS]
   │
   ├─ uniLasso (Python)   github.com/sophial05/uni-lasso
   ├─ uniLasso (R)        CRAN uniLasso
   ├─ AutoGluon           AutoML (boosted trees, NNs, own ensembling)
   └─ lasso               reference that uniLasso is benchmarked against
         │  out-of-fold predictions
         ▼
   non-negative logistic stacker → disease-specific risk score
```

uniLasso (Chatterjee, Hastie & Tibshirani 2025, *Univariate-Guided Sparse Regression*) keeps each selected score's sign equal to its univariate association. So a selected PRS for *another* trait is a directly interpretable pleiotropy signal.

The bundled target is the UK Biobank colorectal-cancer cohort in the parent directory (`../UKB_CRC.QC.{bed,bim,fam}`, 3,917 cases / 3,917 controls). Everything is driven by `config/pipeline.yaml`.

Quick start

```
conda env create -f environment.yml && conda activate stackprs
make r-deps          # CRAN uniLasso for the R learner (needs R >= 3.6)
make test            # offline tests, ~10 s

make metadata        # 01  all PGS Catalog scores + leakage evidence (~1
min)
make filter          # 02  leakage / ancestry / phenotype filters -> chunks
make run JOBS=4      # 03  pgsc_calc per chunk, in parallel (Docker
running)
make merge dataset   # 04-05
make quick           # 06  fast smoke run of the ensemble (holdout,
AutoGluon 60 s)
make ensemble        # 06  full nested evaluation + final model
make importance      # 07  coefficients, R-vs-Python agreement, EFO
enrichment
```

On a cluster, replace step 03 with the Slurm array (one task per chunk):

```
python scripts/00_make_samplesheet.py
N=$(ls data/chunks/chunk_*.txt | wc -l)
sbatch --array=0-$((N-1)) scripts/03_run_pgsc_calc.slurm
```

Pipeline

Step	Script	Main outputs (`data/`)
00	`00_make_samplesheet.py`	`samplesheet.csv`, `pgsc_params.env`
01	`01_fetch_metadata.py`	`all_pgs_metadata.csv`, `cache/pgs_scores.json.gz`
02	`02_filter_and_chunk.py`	`pgs_metadata_clean.csv`, `filter_decisions.csv`, `chunks/`
03	`03_run_pgsc_calc.py` (local, parallel) / `.slurm` (HPC)	`results/chunk_*/<sampleset>/{score,match}/`
04	`04_merge_scores.py`	`pgs_feature_matrix.parquet`, `match_summary.csv`
05	`05_build_dataset.py`	`model_dataset.parquet`, `feature_qc.csv`, `feature_scaling.csv`
06	`06_fit_ensemble.py`	`ensemble/cv_metrics.csv`, `ensemble/final/model.pkl`, …
07	`07_importance_pleiotropy.py`	`ensemble/top_scores.csv`, `ensemble/efo_enrichment.csv`

Choosing the pretrained PRS (step 02)

Each score is dropped by the first filter it fails; `data/filter_decisions.csv` records the reason for every score. Numbers below are for the Catalog as of 2026-09 (6,991 scores) and the default config.

Cohort exclusions (UK Biobank leakage, All of Us)

`filters.leakage.exclude_cohorts` lists cohorts whose participants must not have contributed to a score's source GWAS or score training. The default is `[UKB, AllofUs]`:

```
leakage:
  exclude_cohorts: [UKB, AllofUs]   # any Catalog short name works, e.g.
FinnGen, MVP
  member_consortia: {UKB: [GBMI]}
```

- **UKB and All of Us** come with curated name and free-text patterns.
- **Any other Catalog cohort short name** is matched on the structured cohort list.
- **Spelling is forgiving.** "UK Biobank", "All of Us" and "AoU" all resolve, and a name that matches no score prints a warning with suggestions.

UK Biobank (leakage)

A score whose source GWAS or score training used UK Biobank is inflated on a UKB target. Evidence comes in four kinds, all audited against the full Catalog:

Evidence	Scores
UKB listed as a contributing cohort (`name_short == "UKB"`)	3,549
UKB only in the sample free text, e.g. "UKB GWAS (ICD10-based phenotype): C18" or Neale-lab URLs	+210
a consortium that includes UKB (`member_consortia: {UKB: [GBMI]}`)	+1
no cohort information at all (`on_unknown_provenance: drop`)	+523

- **Exact cohort matching.** `UKBS` (UK National Blood Donor Service) and `UKBGS` (UK Breakthrough Generations Study) are *not* UK Biobank. A `"UKB" in name` test would wrongly drop them.
- **Negation handling.** Negated free text ("samples from UK Biobank excluded", "UKB removed", "without UKB") marks the *clean* re-analyses of UKB GWAS, and those scores are kept.
- **Why the free-text layer matters.** 12 colorectal/colon/rectal-cancer PRS trained on UKB mention it only in free text. Of the Catalog's 88 colorectal-cancer PRS, 60 are removed for leakage.
- **Unknown provenance.** Set `on_unknown_provenance: keep` for a less strict run. Only 6 of those scores predate the first UKB genotype release, so their date doesn't help.

All of Us

The Catalog records All of Us consistently as `name_short: AllofUs` ("All of Us Research Program | NIH").

- **125 scores** use it: 100 for score training or tuning, 25 in the source GWAS. Most come from Truong 2024 (93) and White 2025 (17).
- 30 of them also use UKB, so **95 are removed only by this option**.
- No score mentions All of Us only in free text today. The text pattern guards future records. It matches "AoU" in that capitalization only, because all-caps "AOU" also denotes Italian university hospitals in cohort descriptions.
- **Net effect with the default filters is small: 485 → 483 scores.** 93 of those 95 don't report GWAS ancestry, so the ancestry filter would drop them anyway.
- **With `ancestry.on_missing: keep` it removes 92 scores (703 → 611).**

Remove `AllofUs` from `exclude_cohorts` to keep these scores.

Ancestry

`ancestry: {target: [EUR], source: gwas, min_target_fraction: 80}` keeps scores whose source GWAS is at least 80% European. By default it takes the pool from 2,518 to 483. Of the 2,035 scores it removes, 1,617 are 100% East Asian (Taiwan and BioBank Japan GWAS) and 128 don't report GWAS ancestry. Use `source: dev`/`eval` or a list of targets (e.g. `[EUR, MAE]`) to relax it.

Phenotype scope

Resolved through the Catalog's trait ontology (all descendant traits included, exactly like pgsc_calc's `--efo_id`):

`scope`	**Keeps**	**CRC example (default filters)**
`all`	everything that passes the other filters	483
`same`	`target_traits` only (MONDO_0005575 colorectal cancer)	11
`related`	target + `related_traits` (MONDO_0004992 cancer) + `related_categories`	153
`other`	everything except the target trait (pleiotropy-only analysis)	472

The Catalog has **no "Cancer" category**; colorectal cancer is filed under "Digestive system disorder" and "Other disease". Use the ontology root MONDO_0004992 for "all cancers".

Build

There is no build filter. With `--pgs_id`, pgsc_calc downloads the Catalog's *harmonised* scoring file for your `target.build`, so the source build is irrelevant.

Computing the scores (step 03)

- **Concurrency.** `03_run_pgsc_calc.py` runs `run.jobs` chunks concurrently, each in its own Nextflow launch and work directory, pinned to `run.pipeline_version` (pgsc_calc v2.3.0).
- **Resuming.** Results are stamped with the PGS ids they were computed for. After changing filters the runner refuses to reuse stale results.
- **Minimum overlap.** pgsc_calc drops scores that match fewer than `run.min_overlap` (0.75) of their variants. If *no* score in a chunk passes, the chunk is reported as "no scores passed".
- **Apple Silicon.** On ARM hosts the runner adds pgsc_calc's `arm` profile to `docker` automatically. Without it, pgsc_calc's amd64 containers run under emulation and variant matching segfaults.
- **Array data.** The bundled `UKB_CRC.QC` is a ~77k-variant genotyping-array subset, and genome-wide scores match few of its variants. Use imputed genotypes (e.g. HapMap3 variants) rather than lowering `min_overlap`. pgsc_calc's authors explain why lowering it is a bad idea.
- **Paths with spaces.** Nextflow and container mounts may not handle them. Step 00 warns if yours do (this project lives under `PRS model/`); use a symlink if pgsc_calc fails.

Modelling dataset and confounding check (step 05)

Step 05 joins scores to phenotype and covariates. It QCs scores on completeness, variance and pgsc_calc match rate, and z-scores them without using labels.

- **Confounding warning.** It warns when a covariate *on its own* separates cases from controls. In `UKB_CRC` this fires: sex is perfectly matched (AUC 0.500), but **PC3 alone gives AUC 0.886**, and a logistic model on sex + PC1–6 reaches AUC ≈ 0.92. A genetic PC should not know who has colorectal cancer. This is the signature of a case/control batch or ancestry difference (e.g. different arrays or batches, separate QC, controls from another sub-population). PRS correlated with the PCs would then look predictive for the wrong reason.
- **Residualization.** `dataset.residualize_on: [PC1 … PC6]` (enabled) regresses each PRS on the PCs before modelling. The coefficients are saved so new individuals are transformed identically.
- **It is not a fix.** When a PC is nearly a proxy for case status, projecting it out also removes much of the PRS's *genuine* association with the disease. In a validation run with a planted signal, residualization lowered the ensemble AUC from 0.650 to 0.574, although the PCs explained ≤ 2.3% of any PRS's variance. With confounding like this, **raw AUCs are upper bounds (the artefact can inflate them) and residualized AUCs are conservative lower bounds**. Report both (set `residualize_on: []` for the raw run).
- **The actual fix is upstream.** QC cases and controls together, check the genotyping array and batch, and review how controls were sampled.
- **Reading the covariate models.** `covariates_only` and `ensemble_plus_covariates` include the PCs, so on this cohort their AUCs mostly measure the artefact.

The ensemble (step 06)

- **Evaluation** is nested (`ensemble.outer_folds: 5`; `1` = one stratified holdout). Inside each outer training set, `inner_folds` produce out-of-fold predictions for the stacker, then every learner is refit on the whole outer training set. No test individual's label is used to fit anything that predicts them.
- **Stacker.** Logistic regression on the learners' logits with non-negative weights. It is stable when learners are near-duplicates (the two uniLasso implementations), and the weights are readable as "how much each learner contributes".
- **Reported models.** Each learner, `ensemble`, `ensemble_plus_covariates` (stacker plus sex/PCs), `covariates_only`, and `best_single_pgs` (the best off-the-shelf catalog PRS, chosen on training data).
- **Metrics.** AUC with bootstrap CI, plus a paired-bootstrap Δ-AUC versus the ensemble, AUPRC, Brier score and log-loss.
- **Covariates.** By default the learners are genetic-only (`use_covariates: false`), so the ensemble is a pure genetic risk score and covariates enter transparently via `ensemble_plus_covariates`.
- **Runtime.** A full run makes `outer × (inner + 1) + (inner + 1)` fits per learner (36 with the defaults). At N ≈ 7.8k and ~480 scores, one fit took ~2–5 s for either uniLasso and ~4 s for lasso. AutoGluon is bounded by `autogluon.time_limit` (300 s → up to 3 h in total). `make quick` checks the whole path in ~7 minutes on a 12-core laptop.

uniLasso: two implementations, one upstream bug

The Python and R implementations compute **identical** first-stage leave-one-out fits (max difference 9×10^{-15}), and their second-stage solvers (adelie and glmnet) agree to 10^{-8} at any fixed λ.

The Python package's `extract_cv` (at the pinned commit `c184540`) returns the wrong model, though. After cross-validation, adelie refits along a *new* path that ends at the best λ, and `extract_cv` indexes that path with the *old* path's `best_idx`. With weak PRS-like effects the returned model sits at a several-fold larger λ: on this cohort 3 PGS instead of 16, and AUC 0.636 instead of 0.653. The object still reports the right `best_lmda`, so nothing looks wrong.

`stackprs.learners.UniLassoPy` takes the refit row whose λ equals `best_lmda` instead. After that, the two implementations select the same scores (Jaccard 0.94–1.00 across CV seeds), which `unilasso_agreement.json` reports on every run. `tests/test_learners.py` contains regression tests that fail on the upstream behaviour.

Using the final risk score

```
import pandas as pd
from stackprs.ensemble import StackedModel

model = StackedModel.load("data/ensemble/final/model.pkl")    # AutoGluon
reloads from final/autogluon/
raw = ...   # DataFrame of *raw* pgsc_calc SUM scores for new people,
columns = PGS accessions
cov = ...   # their covariates (Sex, PC1..PC6), same definitions as in
training
X = model.standardize(raw, Z=cov)       # training mean/sd (+ PC
residualisation); missing -> mean
risk = model.predict_proba(X)           # genetic-only ensemble
risk_adj = model.predict_proba(X, Z=cov)  # + sex/PCs (stacker with
covariates)
```

Interpretation (step 07)

- `top_scores.csv`: per PGS, the uniLasso (Python, R) and lasso coefficients, univariate AUC, AutoGluon permutation importance (from held-out folds), trait metadata and target/related-trait flags.
- `unilasso_agreement.json`: selected-set overlap (Jaccard), coefficient rank correlation and sign agreement between the two uniLasso implementations.
- `efo_enrichment.csv`: hypergeometric enrichment of EFO trait labels among each learner's selected PGS against all PGS in the model, with BH FDR. This is the pleiotropy readout.
- `figures/top_scores.png`.

Tests

`make test` runs offline in about 10 s. It covers:

- the leakage patterns, taken from real Catalog records;
- pgsc_calc's output formats: long format, `_hmPOS_` IDs, reference-panel rows, mixed ID types, match summaries;
- the filter funnel and phenotype scopes;
- the parallel runner, against a stub `nextflow`;

- train/inference consistency of the feature transform;
- both uniLasso implementations, including the `extract_cv` regression;
- the stacker and the nested CV.

R and AutoGluon tests skip themselves when the software is missing; `STACKPRS_TEST_AUTOGLUON=1` enables the slow AutoGluon test.

Layout

```
config/      pipeline.yaml, spartan.config (HPC), samplesheet.csv
(template)
stackprs/    catalog.py (API, leakage, traits) · data.py (pgsc_calc IO) ·
             learners.py · ensemble.py · r/unilasso_fit.R
scripts/     00–07 pipeline stages
tests/       pytest suite
data/        generated artefacts (gitignored)
```